

Pythonia

Manual de estudio: tronco común

1

Explorador

Nivel 1: Principiante

De cero a tu primer programa útil.

11 lecciones | 6 semanas | 1 proyecto integrador

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Entender cómo piensa Python: valores, decisiones, repeticiones y funciones. Al terminar podrás escribir programas pequeños que conversan con el usuario.

Cada lección tiene la misma estructura: explicación breve, un ejemplo para escribir y ejecutar, la salida real que produce, consejos, un ejercicio y una autoevaluación (respuestas al final). Al terminar este nivel harás la **degustación** de las 13 rutas en la app y el test vocacional para elegir tu ruta principal.

Además, cada lección tiene tres apartados de profundización (**el concepto a fondo**, **el ejemplo línea por línea** y **errores frecuentes**) y una **práctica evaluable**: un ejercicio que escribes en la app y se corrige automáticamente con pruebas. Cada práctica es una pieza del proyecto del nivel, que se va armando solo a medida que avanzas.

Método: lee en la app, escribe el ejemplo a mano, predice la salida, ejecútalo y anota el resultado en una ficha de **Registro de ejecución** del Cuadernillo-Diario. Cuando apruebes la práctica, regístrala en la **Bitácora de prácticas**: intentos, pistas usadas, el error que más te costó y qué aprendiste.

Tu plan

Semana	Temas	Misión de la semana
1	1. Tu primer programa 2. Variables y tipos de datos 3. Operadores	Dado un total de 125 minutos, calcula cuántas horas y minutos son usando // y %.
2	4. Cadenas de texto (strings) 5. Entrada del usuario y conversiones	Crea una calculadora de propinas: pide el monto de la cuenta y el porcentaje, y muestra la propina y el total.
3	6. Condicionales 7. Bucles for y while	Imprime la tabla de multiplicar de un número que elija el usuario, del 1 al 10.
4	8. Listas y tuplas 9. Diccionarios y conjuntos	Cuenta cuántas veces aparece cada palabra en una frase usando un diccionario.
5	10. Funciones 11. Errores y cómo leerlos	Mejora la calculadora de propinas para que vuelva a pedir el monto si el usuario escribe algo inválido.
6	Proyecto integrador: Adivina el número	Construir, probar y documentar el proyecto

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

1 Tu primer programa

Un programa es una lista de instrucciones que la computadora ejecuta de arriba hacia abajo. En Python, la instrucción más básica es `print()`, que muestra en pantalla lo que le pases.

Los comentarios empiezan con `#` y Python los ignora: sirven para explicarle a otras personas (y a tu yo del futuro) qué hace el código.

`print()` acepta varios valores separados por comas. Por defecto los une con un espacio, pero puedes cambiar el separador con `sep` y el final de línea con `end`.

El concepto a fondo

Python lee tu archivo de arriba hacia abajo, una línea a la vez. Cada línea es una instrucción. Si una línea tiene un error, el programa se detiene ahí y te avisa en qué línea fue: por eso conviene escribir poco, ejecutar y seguir.

`print()` es una función: un nombre seguido de paréntesis. Lo que va dentro de los paréntesis son sus argumentos. Puedes pasarle varios valores separados por comas y los imprime separados por un espacio. Ese espacio se puede cambiar con `sep`, y el salto de línea final con `end`.

Los textos van entre comillas, dobles o simples, siempre del mismo tipo al abrir y al cerrar. Lo que está entre comillas Python no lo interpreta: lo muestra tal cual. Todo lo que va después de `#` es un comentario: Python lo ignora y sirve para explicar tu código a otras personas (y a ti dentro de un mes).

Ejemplo

```
1 # Mi primer programa
2 print("¡Hola, Pythonia!")
3 print("Python", "es", "genial", sep="-")
4 print("Sin salto", end=" ... ")
5 print("¡y sigo en la misma línea!")
```

Salida real

```
¡Hola, Pythonia!
Python-es-genial
Sin salto ... ¡y sigo en la misma línea!
```

El ejemplo, línea por línea

1. `# Mi primer programa` es un comentario: no se ejecuta.
2. `print("¡Hola, Pythonia!")` muestra el texto y salta a la línea siguiente.
3. `sep="-"` cambia el separador entre los tres textos: el espacio pasa a ser un guion.
4. `end=" ... "` reemplaza el salto de línea final, así que el siguiente `print` continúa en la misma línea.

Consejos

- Escribe y ejecuta código todos los días, aunque sean 10 minutos.
- Si algo falla, lee el mensaje de error completo: casi siempre te dice qué pasó y en qué línea.

Ejercicio

Escribe un programa que imprima tu nombre, tu ciudad y tu comida favorita, cada uno en una línea, y luego los tres juntos separados por ' | '.

Errores frecuentes

Error frecuente	Cómo resolverlo
SyntaxError: unterminated string literal	Abriste comillas y no las cerraste. Revisa que cada texto tenga sus dos comillas.
NameError: name 'Print' is not defined	Python distingue mayúsculas de minúsculas: la función se llama print, en minúscula.

Práctica evaluable**La bienvenida del juego** · Pieza del proyecto: **Capítulo 1: la bienvenida**

Tu proyecto del Nivel 1 es el juego ****Adivina el número****. Empieza por la pantalla de bienvenida: imprime exactamente estas cuatro líneas. La última debe hacerse con UN solo print que use sep=" | ".

```
=== ADIVINA EL NÚMERO ===
Pensé un número del 1 al 100.
¿Podrás descubrirlo?
Más alto | Más bajo | ¡Correcto!
```

Plantilla para empezar

```
1 # Capítulo 1 de tu proyecto: la bienvenida del juego.
2 # 1) El título (ya está hecho como ejemplo):
3 print("=== ADIVINA EL NÚMERO ===")
4 # 2) Imprime: Pensé un número del 1 al 100.
5
6 # 3) Imprime: ¿Podrás descubrirlo?
7
8 # 4) Imprime las tres pistas en UNA línea usando sep=" | "
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

print("A", "B", sep="") muestra...

- a) AB
- b) A,B
- c) A B
- d) Error

2 Variables y tipos de datos

Una variable es un nombre que apunta a un valor. Se crea con `=`: `edad = 30`. No hace falta declarar el tipo; Python lo deduce.

Los tipos básicos son: `int` (enteros), `float` (decimales), `str` (texto) y `bool` (`True/False`). Con `type()` puedes preguntar el tipo de cualquier valor.

Las f-strings (`f"..."`) permiten insertar variables dentro de un texto usando llaves: es la forma moderna y más legible de armar mensajes.

El concepto a fondo

Una variable es una etiqueta que pegas a un valor. `edad = 30` no significa "edad es igual a 30" como en matemática, sino "guarda 30 y llámalo edad". Si después escribes `edad = 31`, la etiqueta se mueve al nuevo valor.

Cada valor tiene un tipo: `int` (enteros), `float` (con decimales), `str` (texto) y `bool` (`True` o `False`). El tipo decide qué puedes hacer: sumar dos `int` da un número, pero sumar dos `str` las pega. `type(x)` te dice el tipo de cualquier valor.

Las f-strings son la forma moderna de mezclar texto y variables: pon una `f` antes de las comillas y escribe las variables entre llaves, `f"Tengo {edad} años"`. Dentro de las llaves puede ir cualquier expresión, como `{edad + 1}`.

Elige nombres que expliquen qué guardan: `precio_total` es mejor que `pt` o `x`. En Python se usa `snake_case`: minúsculas y palabras separadas por guion bajo. Un nombre no puede empezar con un número ni tener espacios.

Ejemplo

```
1 nombre = "Luna"
2 edad = 30
3 altura = 1.65
4 estudia = True
5
6 print(type(edad), type(altura), type(nombre), type(estudia))
7 print(f"{nombre} tiene {edad} años y mide {altura} m")
```

Salida real

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
Luna tiene 30 años y mide 1.65 m
```

El ejemplo, línea por línea

1. Se crean cuatro variables, una de cada tipo básico.
2. `type(edad)` devuelve `<class 'int'>`: el tipo del valor guardado.
3. La f-string reemplaza `{nombre}`, `{edad}` y `{altura}` por sus valores.

Consejos

- Usa nombres descriptivos en minúsculas con guion bajo: `precio_total`, no `pt`.
- Python distingue mayúsculas: `Edad` y `edad` son variables distintas.

Ejercicio

Crea variables para un producto (nombre, precio, cantidad, disponible) e imprime una ficha usando una f-string.

Errores frecuentes

Error frecuente	Cómo resolverlo
NameError: name 'edda' is not defined	Escribiste mal el nombre de una variable o la usaste antes de crearla.
TypeError: can only concatenate str (not "int") to str	Intentaste sumar texto y número con +. Usa una f-string o conviértelo con str().

Práctica evaluable**La configuración del juego** · Pieza del proyecto: **Capítulo 2: la configuración**

Crea las variables de configuración del juego: nombre_juego con el texto "Adivina el número", mínimo con 1, máximo con 100, intentos_max con 7 y con_pistas con True. Después imprime, usando una f-string con esas variables:

Adivina el número: del 1 al 100 en 7 intentos

Plantilla para empezar

```

1 # Capítulo 2: la configuración del juego.
2 nombre_juego = "Adivina el número"
3 # Crea minimo, maximo e intentos_max (números enteros, sin comillas)
4
5 # Crea con_pistas (un booleano)
6
7 # Imprime con una f-string: Adivina el número: del 1 al 100 en 7 intentos

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué tipo devuelve type(3 / 2)?

- a) str
- b) int
- c) bool
- d) float

3 Operadores

Aritméticos: + - * /, más tres muy útiles: // (división entera), % (resto o módulo) y ** (potencia).

Comparación: == != > < >= <= devuelven siempre un bool. Ojo: = asigna y == compara.

Lógicos: and, or, not combinan condiciones. Python evalúa en cortocircuito: en a and b, si a es falso, ni mira b.

El concepto a fondo

Python respeta el orden matemático: primero **, después *, /, // y %, y al final + y -. Ante la duda, usa paréntesis: hacen el código más claro y nunca están de más.

Hay dos divisiones. / siempre devuelve un float (10 / 2 da 5.0). // devuelve la parte entera (7 // 2 da 3). Y % devuelve el resto (7 % 2 da 1): es la herramienta para saber si un número es par (n % 2 == 0) o para convertir minutos en horas.

Los operadores de comparación (==, !=, <, >, <=, >=) no calculan números: responden preguntas y devuelven True o False. Cuidado: = guarda un valor y == compara dos valores.

Los operadores lógicos combinan respuestas: and es verdadero solo si ambas partes lo son, or si al menos una lo es, y not invierte. abs(x) es una función integrada que devuelve el valor sin signo: abs(-5) da 5.

Ejemplo

```
1 print(7 / 2, 7 // 2, 7 % 2, 2 ** 3)
2 print(5 > 3 and 2 > 4)
3 print(5 > 3 or 2 > 4)
4 print(not False)
5 print(10 == 10.0)
```

Salida real

```
3.5 3 1 8
False
True
True
True
```

El ejemplo, línea por línea

1. 7 / 2 da 3.5; 7 // 2 da 3; 7 % 2 da 1; 2 ** 3 da 8.
2. 5 > 3 and 2 > 4 es False porque la segunda parte es falsa.
3. 5 > 3 or 2 > 4 es True porque basta una parte verdadera.
4. 10 == 10.0 es True: compara valores, no tipos.

Consejos

- n % 2 == 0 es la forma clásica de saber si un número es par.
- Usa paréntesis para dejar clara la prioridad, aunque no sean obligatorios.

Ejercicio

Dado un total de 125 minutos, calcula cuántas horas y minutos son usando // y %.

Errores frecuentes

Error frecuente	Cómo resolverlo
ZeroDivisionError: division by zero	Dividiste por cero. Revisa que el divisor no pueda valer 0, o protégelo con un if.
SyntaxError al escribir if x = 5	Para comparar se usa ==. Un solo = es para asignar.

Práctica evaluable**Los cálculos del juego** · Pieza del proyecto: **Capítulo 3: los cálculos**

Ya existen las variables minimo, maximo, secreto e intento (el evaluador probará varios valores distintos).

Calcula:

- cantidad: cuántos números posibles hay entre minimo y maximo, ambos incluidos.
- mitad: el número del medio, con división entera.
- distancia: qué tan lejos está el intento del secreto, sin signo.
- es_par: True si el secreto es par.
- acerto: True si el intento es igual al secreto.

Datos que prepara el evaluador (también prueba con otros valores)

```

1 minimo = 1
2 maximo = 100
3 secreto = 42
4 intento = 30

```

Plantilla para empezar

```

1 # Capítulo 3: los cálculos del juego.
2 # Ya existen: minimo, maximo, secreto e intento.
3 # 1) cantidad (ya está hecho como ejemplo):
4 cantidad = maximo - minimo + 1
5 # 2) mitad: (minimo + maximo) dividido con división entera //
6
7 # 3) distancia: usa abs() para quitar el signo a secreto - intento
8
9 # 4) es_par: el resto de dividir secreto por 2 es 0
10
11 # 5) acerto: compara intento con secreto usando ==

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuánto vale 10 % 3?

- a) 3.33
- b) 1
- c) 3
- d) 0

4 Cadenas de texto (strings)

Un string es una secuencia de caracteres. Cada carácter tiene un índice que empieza en 0; los índices negativos cuentan desde el final (-1 es el último).

El slicing `s[inicio:fin]` extrae un trozo: incluye inicio y excluye fin. `s[::-1]` invierte el texto.

Los strings tienen métodos: `.upper()`, `.lower()`, `.strip()`, `.replace()`, `.split()`, `.startswith()`... Son inmutables: los métodos devuelven un string nuevo, no modifican el original.

El concepto a fondo

Un string es una secuencia de caracteres, y cada carácter tiene una posición (índice) que empieza en 0. `s[0]` es el primero y `s[-1]` el último. Un rebanado (slice) `s[inicio:fin]` toma desde inicio hasta antes de fin; si omite uno de los dos, llega hasta el borde.

Los strings son inmutables: ningún método los modifica, todos devuelven un string nuevo. Por eso `nombre.upper()` no cambia `nombre`; si quieres conservar el resultado, guárdalo: `nombre = nombre.upper()`.

Los métodos más útiles para limpiar datos que escribe una persona: `strip()` quita espacios de los bordes, `lower()` y `upper()` cambian a minúsculas y mayúsculas, `title()` pone mayúscula inicial en cada palabra, `replace(a, b)` reemplaza y `split()` corta en una lista de palabras.

Puedes encadenar métodos, porque cada uno devuelve un string: `" ana ".strip().title()` primero limpia los espacios y luego da "Ana". `len(s)` cuenta los caracteres, incluidos los espacios.

Ejemplo

```
1 s = "Pythonia"
2 print(s[0], s[-1], s[0:3])
3 print(s.upper(), len(s))
4 print(s.replace("ia", "!"))
5 print(s[::-1])
6 print("a,b,c".split(","))
```

Salida real

```
P a Pyt
PYTHONIA 8
Python!
ainohtyP
['a', 'b', 'c']
```

El ejemplo, línea por línea

1. `s[0]` es 'P', `s[-1]` es 'a' y `s[0:3]` es 'Pyt' (posiciones 0, 1 y 2).
2. `upper()` devuelve una copia en mayúsculas; `len(s)` cuenta 8 caracteres.
3. `replace("ia", "!")` cambia 'ia' por '!'.
`Pythonia` se convierte en `Python!`.
4. `s[::-1]` recorre al revés con paso -1.
`Pythonia` se convierte en `ainohtyP`.
5. `split(",")` corta el texto en cada coma.
`"a,b,c"` se convierte en `['a', 'b', 'c']`.

Consejos

- `.strip()` es tu amigo al leer datos: elimina espacios y saltos de línea sobrantes.
- Para unir una lista de textos usa `", ".join(lista)`.

Ejercicio

Pide una frase y muestra: cantidad de caracteres, la frase en mayúsculas, la frase al revés y cuántas palabras tiene.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>IndexError: string index out of range</code>	Pediste una posición que no existe. Un texto de 5 letras tiene índices del 0 al 4.
<code>AttributeError: 'int' object has no attribute 'upper'</code>	Usaste un método de texto sobre un número. Revisa el tipo de la variable.

Práctica evaluable**Los mensajes del juego** · Pieza del proyecto: **Capítulo 4: los mensajes**

El nombre llega desprolijo, con espacios y mayúsculas mezcladas, en `jugador_crudo`. Ya existen también `numero_intento` e `intentos_max`. Crea:

- `jugador`: el nombre sin espacios en los bordes y con mayúscula inicial en cada palabra.
- `mensaje`: "Ana, intento 3 de 7" (con los valores reales).
- `inicial`: la primera letra de `jugador`.
- `grito`: el mensaje en mayúsculas.
- `largo`: la cantidad de caracteres de `jugador`.

Datos que prepara el evaluador (también prueba con otros valores)

```
1 jugador_crudo = " ana "
2 numero_intento = 3
3 intentos_max = 7
```

Plantilla para empezar

```
1 # Capítulo 4: los mensajes. Ya existen jugador_crudo, numero_intento e intentos_max.
2 # 1) jugador: limpia con strip() y luego title()
3 jugador = jugador_crudo.strip()
4 # 2) mensaje: una f-string como "Ana, intento 3 de 7"
5
6 # 3) inicial: la letra en la posición 0 de jugador
7
8 # 4) grito: el mensaje con upper()
9
10 # 5) largo: usa len()
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué devuelve "hola"[1:3]?

- a) "ola"
- b) "ho"
- c) "ol"
- d) "hol"

5**Entrada del usuario y conversiones**

`input("mensaje")` muestra un mensaje, espera a que el usuario escriba y devuelve lo escrito como `str`, siempre.

Para hacer cuentas necesitas convertir: `int()`, `float()`. Para volver a texto: `str()`.

Si el usuario escribe algo que no es número, `int()` lanza un error. En la lección de errores verás cómo manejarlo con elegancia.

El concepto a fondo

`input()` siempre devuelve texto, aunque la persona escriba un número. Para hacer cuentas hay que convertir: `int("57")` da el entero 57 y `float("3.5")` da el decimal 3.5. Al revés, `str(57)` convierte un número en texto.

Si olvidas convertir, Python no siempre da error: `"57" * 2` da `"5757"` (repite el texto) y `"57" + "1"` da `"571"` (los pega). Estos errores silenciosos son de los más difíciles de detectar, por eso conviene convertir apenas recibes el dato.

`int()` tolera espacios en los bordes (`int(" 57 ")` funciona), pero no acepta letras ni decimales: `int("hola")` y `int("3.5")` lanzan `ValueError`. En el Capítulo 11 aprenderás a atrapar ese error para que el programa no se caiga.

En la práctica de esta lección el texto ya viene preparado en una variable, como si la persona lo hubiera escrito. En tu IDE lo reemplazarás por `texto = input("Tu número: ")`: la conversión es exactamente la misma.

Ejemplo

```
1 edad = int(input("¿Tu edad? ")) # el usuario escribe 20
2 print(f"En 5 años tendrás {edad + 5}")
```

Salida real

```
¿Tu edad? 20
En 5 años tendrás 25
```

El ejemplo, línea por línea

1. `input("¿Tu edad? ")` muestra la pregunta y espera; devuelve lo escrito como texto.
2. `int(...)` convierte ese texto en número entero.
3. La f-string calcula `edad + 5` dentro de las llaves.

Consejos

- Valida siempre la entrada del usuario: nunca confíes en que escribirá lo esperado.
- `input().strip()` evita problemas con espacios accidentales.

Ejercicio

Crea una calculadora de propinas: pide el monto de la cuenta y el porcentaje, y muestra la propina y el total.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>ValueError: invalid literal for int() with base 10: 'hola'</code>	Intentaste convertir algo que no es un número entero. Valida antes o usa <code>try/except</code> (lección 11).
Resultado '5757' en lugar de 114	Multiplicaste un texto. Convierte con <code>int()</code> antes de operar.

Práctica evaluable**Convertir lo que escribe el jugador** · Pieza del proyecto: **Capítulo 5: la entrada del jugador**

En el juego real, el número llega desde `input()` y por eso es texto. Aquí ya existe la variable `texto` (por ejemplo "57"). Crea:

- `intento`: el texto convertido a número entero.
- `doble`: el intento multiplicado por 2.
- `siguiente`: el intento más 1.
- `resumen`: el texto "Elegiste el 57" (con el número real), uniendo con `+` y `str()`.

Datos que prepara el evaluador (también prueba con otros valores)

```
1 texto = " 57 "
```

Plantilla para empezar

```
1 # Capítulo 5: lo que escribe el jugador llega como TEXT0.
2 # Ya existe la variable texto, por ejemplo " 57 ".
3 # 1) intento: convierte texto a entero con int()
4
5 # 2) doble: intento por 2
6
7 # 3) siguiente: intento más 1
8
9 # 4) resumen: "Elegiste el " + el número convertido con str()
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué tipo de dato devuelve `input()`?

- a) float
- b) depende de lo que escriba
- c) str
- d) int

6 Condicionales

`if` ejecuta un bloque solo si la condición es verdadera. `elif` agrega más casos y `else` cubre todo lo demás. Python solo ejecuta el primer bloque cuya condición se cumpla.

La indentación (4 espacios) no es estética: define qué líneas pertenecen a cada bloque.

Valores `'falsy'`: `0`, `0.0`, `""`, `[]`, `{}`, `None` y `False` se comportan como falso en un `if`. Todo lo demás es `'truthy'`.

El concepto a fondo

Un `if` evalúa una condición y ejecuta su bloque solo si es verdadera. El bloque se reconoce por la sangría (cuatro espacios): todo lo que está indentado debajo del `if` le pertenece. Los dos puntos al final de la línea del `if` son obligatorios.

Con `elif` agregas más casos, que se revisan en orden: apenas uno se cumple, los siguientes se saltan. `else` atrapa todo lo que no cumplió ninguna condición. Por eso el orden importa: pon primero los casos más específicos.

Las condiciones se pueden combinar con `and`, `or` y `not`, y Python permite comparaciones encadenadas como `0 < x <= 5`, que se lee igual que en matemática.

Para decisiones cortas existe la expresión condicional: `valor = "par" if n % 2 == 0 else "impar"`. Úsala solo cuando sea fácil de leer; si la condición es compleja, un `if` normal es más claro.

Ejemplo

```
1  nota = 78
2  if nota >= 90:
3      print("Excelente")
4  elif nota >= 70:
5      print("Aprobado")
6  else:
7      print("A repasar")
8
9  mensaje = "par" if nota % 2 == 0 else "impar"
10 print(mensaje)
```

Salida real

```
Aprobado
par
```

El ejemplo, línea por línea

1. `nota >= 90` es falso, así que se pasa al `elif`.
2. `nota >= 70` es verdadero: imprime 'Aprobado' y saltea el `else`.
3. La expresión condicional elige 'par' o 'impar' según el resto de dividir por 2.

Consejos

- Ordena los `elif` del caso más específico al más general.
- El operador ternario `x if cond else y` es ideal para asignaciones cortas.

Ejercicio

Clasifica una temperatura: menos de 10 'frío', de 10 a 25 'templado', más de 25 'calor'.

Errores frecuentes

Error frecuente	Cómo resolverlo
IndentationError: expected an indented block	Después de los dos puntos del <code>if</code> el bloque debe ir indentado con 4 espacios.
Siempre entra al primer caso	Revisa el orden: si pones <code>nota >= 70</code> antes que <code>nota >= 90</code> , el 95 nunca llegará a 'Excelente'.

Práctica evaluable**Las pistas del juego** · Pieza del proyecto: **Capítulo 6: las pistas**

Ya existen `secreto` e `intento`. Crea:

- `pista`: "Más alto" si el intento es menor que el secreto, "Más bajo" si es mayor, y "¡Correcto!" si son iguales.
- `cerca`: True si el intento NO es el secreto pero está a 5 o menos de distancia; False en cualquier otro caso.

Datos que prepara el evaluador (también prueba con otros valores)

```
1 secreto = 42
2 intento = 30
```

Plantilla para empezar

```
1 # Capítulo 6: las pistas. Ya existen secreto e intento.
2 if intento < secreto:
3     pista = "Más alto"
4 # Completa con elif ("Más bajo") y else ("¡Correcto!")
5
6
7 # cerca: True si la distancia está entre 1 y 5 (incluidos)
8 distancia = abs(secreto - intento)
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

```
x = 0
```

```
if x: print("A")
```

```
else: print("B")
```

¿Qué imprime?

a) A

b) Error

c) Nada

d) B

7 Bucles for y while

`for` recorre cualquier secuencia (listas, strings, rangos). `range(inicio, fin, paso)` genera números sin incluir `fin`.

`while` repite mientras una condición sea verdadera. Asegúrate de que algo cambie dentro del bucle o se volverá infinito.

`break` sale del bucle inmediatamente; `continue` salta a la siguiente vuelta.

El concepto a fondo

Un bucle repite un bloque. `for` recorre una secuencia elemento por elemento: una lista, un texto o un `range`. `range(inicio, fin, paso)` genera números desde `inicio` hasta antes de `fin`; `range(5)` da 0, 1, 2, 3 y 4.

`while` repite mientras una condición sea verdadera. Es ideal cuando no sabes de antemano cuántas vueltas harán falta, como en un juego que termina cuando se acierta. Dentro del bloque algo debe cambiar para que la condición alguna vez sea falsa; si no, el bucle es infinito.

`break` sale del bucle inmediatamente y `continue` saltea el resto de la vuelta actual y pasa a la siguiente. Úsalos con moderación: un bucle con una condición clara suele leerse mejor.

Patrón muy común: un acumulador o contador que se crea antes del bucle (`pasos = 0`) y se actualiza en cada vuelta (`pasos += 1`). `x += 1` es un atajo de `x = x + 1`.

Ejemplo

```
1 for i in range(1, 6):
2     if i == 3:
3         continue
4     print(i, end=" ")
5 print()
6
7 n = 3
8 while n > 0:
9     print("Despegue en", n)
10    n -= 1
11 print("¡Despegue!")
```

Salida real

```
1 2 4 5
Despegue en 3
Despegue en 2
Despegue en 1
¡Despegue!
```

El ejemplo, línea por línea

1. `range(1, 6)` genera del 1 al 5.
2. Cuando `i` vale 3, `continue` saltea el `print` de esa vuelta.
3. `end=" "` mantiene los números en una línea; el `print()` vacío cierra la línea.
4. El `while` descuenta `n` hasta que `n > 0` deja de cumplirse.

Consejos

- `enumerate(lista)` te da índice y valor a la vez: `for i, x in enumerate(lista):`.
- Si un programa no termina, deténlo con `Ctrl + C`.

Ejercicio

Imprime la tabla de multiplicar de un número que elija el usuario, del 1 al 10.

Errores frecuentes

Error frecuente	Cómo resolverlo
El programa no termina	Bucle infinito: la variable de la condición nunca cambia. En la app, el evaluador lo corta a las 100.000 vueltas; en tu IDE, deténlo con <code>Ctrl+C</code> .
Faltan o sobran vueltas	Recuerda que <code>range(1, 10)</code> llega hasta el 9: el valor final no se incluye.

Práctica evaluable**La computadora adivina** · Pieza del proyecto: **Capítulo 7: la computadora juega**

¡Ahora la computadora juega a tu juego! Usará la mejor estrategia: probar siempre el número del medio del rango posible y descartar la mitad que no sirve. Ya existen `secreto`, `minimo` y `maximo`.

Completa el bucle `while` para que en cada vuelta imprima "Pruebo N" y actualice `bajo`, `alto` o `adivinado`. Al terminar, `pasos` debe tener la cantidad de intentos y `adivinado` el `secreto`.

(Con esta estrategia, cualquier número del 1 al 100 se encuentra en 7 pasos o menos.)

Datos que prepara el evaluador (también prueba con otros valores)

```
1 secreto = 42
2 minimo = 1
3 maximo = 100
```

Plantilla para empezar

```
1 # Capítulo 7: ¡la computadora juega! Estrategia: partir el rango por la mitad.
2 bajo = minimo
3 alto = maximo
4 pasos = 0
5 adivinado = None
6 while adivinado != secreto:
7     medio = (bajo + alto) // 2
8     pasos = pasos + 1
9     print("Pruebo", medio)
10    # Si medio es MENOR que el secreto, el secreto está arriba: bajo = medio + 1
11    # Si medio es MAYOR, está abajo: alto = medio - 1
12    # Si es IGUAL, ¡lo encontró!: adivinado = medio
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuántos números genera `range(2, 10, 2)`?

- a) 8
- b) 5
- c) 3
- d) 4

8 Listas y tuplas

Una lista guarda varios valores en orden y es mutable: puedes agregar (`append`, `insert`), quitar (`remove`, `pop`) y ordenar (`sort`).

Las listas usan los mismos índices y slicing que los strings.

Una tupla es como una lista pero inmutable: `(3, 4)`. Se usan para datos que no deben cambiar y permiten desempaquetar: `x, y = punto`.

El concepto a fondo

Una lista guarda varios valores en orden, entre corchetes: `[50, 25, 37]`. Se accede con índices igual que en los strings, y también admite rebanados: `lista[-3:]` son los tres últimos.

A diferencia de los strings, las listas son mutables: `append` agrega al final, `insert` en una posición, `remove` quita por valor y `pop` por posición. `lista.sort()` ordena la propia lista; `sorted(lista)` devuelve una copia ordenada y deja la original intacta.

Funciones integradas que trabajan con listas: `len`, `min`, `max`, `sum` y `sorted`. `enumerate(lista)` entrega pares (posición, valor) y es la forma pythónica de numerar elementos.

Una tupla es como una lista pero inmutable, y se escribe con paréntesis: `(25, 50)`. Se usa para datos que van juntos y no deben cambiar, como coordenadas o un mínimo y un máximo. Se puede desempaquetar: `x, y = punto`.

Ejemplo

```
1  frutas = ["mango", "pera"]
2  frutas.append("kiwi")
3  frutas.insert(0, "uva")
4  print(frutas, len(frutas))
5  print(frutas[1:])
6  print(sorted(frutas))
7
8  punto = (3, 4)
9  x, y = punto
10 print(x + y)
```

Salida real

```
['uva', 'mango', 'pera', 'kiwi'] 4
['mango', 'pera', 'kiwi']
['kiwi', 'mango', 'pera', 'uva']
7
```

El ejemplo, línea por línea

1. `append` agrega 'kiwi' al final e `insert(0, ...)` pone 'uva' al principio.
2. `frutas[1:]` es la lista desde la posición 1.
3. `sorted` devuelve una copia ordenada alfabéticamente.
4. `x, y = punto` desempaqueta la tupla en dos variables.

Consejos

- `lista.sort()` ordena la lista original; `sorted(lista)` devuelve una nueva.
- `x in lista` verifica si un elemento existe.

Ejercicio

Crea una lista de compras, agrega 3 productos, elimina uno, muéstrala ordenada y numerada con `enumerate`.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>IndexError: list index out of range</code>	La lista tiene menos elementos de los que crees. Revisa <code>len(lista)</code> .
La lista original quedó desordenada	Usaste <code>lista.sort()</code> , que modifica la lista. Si necesitas conservarla, usa <code>sorted(lista)</code> .

Práctica evaluable

El historial de intentos · Pieza del proyecto: Capítulo 8: el historial

La lista `historial` guarda los intentos de una partida, en orden. Crea:

- `cantidad`: cuántos intentos hubo.
- `primero y ultimo`: el primer y el último intento.
- `ultimos_tres`: una lista con los tres últimos.
- `ordenados`: los intentos de menor a mayor, SIN modificar `historial`.
- `extremos`: una tupla (mínimo, máximo).

Datos que prepara el evaluador (también prueba con otros valores)

```
1 historial = [50, 25, 37, 31, 34]
```

Plantilla para empezar

```
1 # Capítulo 8: el historial. Ya existe la lista historial.
2 cantidad = len(historial)
3 # primero y ultimo: posiciones 0 y -1
4
5 # ultimos_tres: un rebanado con índice negativo
6
7 # ordenados: usa sorted() para no tocar historial
8
9 # extremos: una tupla con min() y max()
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

```
t = (1, 2)
```

```
t[0] = 9
```

¿Qué pasa?

- a) Nada
- b) t queda (1, 2, 9)
- c) TypeError
- d) t queda (9, 2)

9**Diccionarios y conjuntos**

Un diccionario asocia claves con valores: {"cafe": 2.5}. Se accede con `d[clave]` y se recorre con `.items()`, `.keys()` o `.values()`.

`d.get(clave, defecto)` evita errores si la clave no existe.

Un conjunto (set) guarda elementos únicos sin orden. Es perfecto para eliminar duplicados y hacer operaciones como unión `|` e intersección `&`.

El concepto a fondo

Un diccionario asocia claves con valores: {"Ana": 5}. Buscar por clave es instantáneo, aunque tenga millones de entradas. `d[clave] = valor` crea o actualiza; `d[clave]` lee y falla con `KeyError` si no existe, mientras que `d.get(clave, defecto)` devuelve un valor por defecto.

`clave in d` pregunta si la clave existe. Para recorrer: `for clave in d` da las claves, `d.values()` los valores y `d.items()` los pares (clave, valor), que se desempaquetan en el `for`.

Un conjunto (set) guarda valores únicos y sin orden: {10, 20, 10} queda como {10, 20}. Es perfecto para eliminar repetidos y para operaciones de conjuntos: unión `|`, intersección `&` y diferencia `-`.

Truco útil: para saber si una lista tiene repetidos, compara `len(set(lista))` con `len(lista)`. Si el conjunto es más chico, algo se repetía.

Ejemplo

```
1 precios = {"cafe": 2.5, "te": 1.8}
2 precios["jugo"] = 3.0
3 for prod, p in precios.items():
4     print(prod, p)
5 print(precios.get("agua", "no hay"))
6
7 colores = {"rojo", "azul", "rojo"}
8 print(len(colores))
9 print({1, 2, 3} & {2, 3, 4})
```

Salida real

```
cafe 2.5
te 1.8
jugo 3.0
no hay
2
{2, 3}
```

El ejemplo, línea por línea

1. Se agrega la clave 'jugo' con su precio.
2. `.items()` entrega cada par producto, precio.
3. `get("agua", "no hay")` evita el error si la clave no existe.
4. El set elimina el 'rojo' repetido; & da los elementos comunes.

Consejos

- Las claves deben ser inmutables (str, int, tuplas).
- Desde Python 3.7 los diccionarios conservan el orden de inserción.

Ejercicio

Cuenta cuántas veces aparece cada palabra en una frase usando un diccionario.

Errores frecuentes

Error frecuente	Cómo resolverlo
KeyError: 'Eva'	Leíste una clave que no existe. Pregunta antes con <code>in</code> o usa <code>.get()</code> .
TypeError: unhashable type: 'list'	Una lista no puede ser clave de diccionario ni elemento de un set. Usa una tupla.

Práctica evaluable

Récords y repetidos · Pieza del proyecto: Capítulo 9: los récords

`records` guarda cuántos intentos necesitó cada jugador en su mejor partida. Acaba de jugar jugador y necesitó intentos. Además, `historial` tiene los números que probó.

- Actualiza `records`: guarda el resultado si el jugador es nuevo o si mejoró (menos intentos que antes).
- `mejor`: el nombre con menos intentos (recorre con un `for`).
- `repetidos`: True si en `historial` hay algún número repetido.
- `unicos`: los números de `historial` sin repetir, ordenados.

Datos que prepara el evaluador (también prueba con otros valores)

```
1 records = {"Ana": 5, "Luis": 3}
2 jugador = "Eva"
3 intentos = 4
4 historial = [10, 20, 10, 30]
```

Plantilla para empezar

```

1 # Capítulo 9: récords. Ya existen records, jugador, intentos e historial.
2 # 1) Guarda si es nuevo 0 si mejoró
3 if jugador not in records or intentos < records[jugador]:
4     records[jugador] = intentos
5
6 # 2) mejor: recorre records.items() buscando el menor número de intentos
7 mejor = None
8 for nombre, n in records.items():
9     pass # reemplaza pass por un if que actualice mejor
10
11 # 3) repetidos: compara len(set(historial)) con len(historial)
12
13 # 4) unicos: sorted(set(...))

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuánto vale `len({1, 1, 2, 3, 3})`?

- a) 2
- b) 3
- c) 5
- d) Error

10 Funciones

Una función agrupa código reutilizable. Se define con `def`, recibe parámetros y devuelve un resultado con `return`.

Los parámetros pueden tener valores por defecto y pasarse por nombre, lo que hace las llamadas más claras.

El docstring (texto entre triples comillas justo después del `def`) documenta qué hace la función. Si una función no tiene `return`, devuelve `None`.

El concepto a fondo

Una función es un bloque con nombre que puedes usar muchas veces. Se define con `def nombre(parámetros):` y el cuerpo va indentado. `return` devuelve un resultado y termina la función; si no hay `return`, la función devuelve `None`.

Los parámetros son las variables que recibe la función; los argumentos, los valores que le pasas al llamarla. Un parámetro con valor por defecto (`maximo=100`) se vuelve opcional. Al llamar puedes nombrar los argumentos: `es_valido(5, maximo=10)`.

Las variables creadas dentro de una función son locales: existen solo mientras la función corre. Por eso una función bien hecha recibe todo lo que necesita por parámetros y entrega su resultado con `return`, sin depender de variables de afuera.

Diferencia clave: `print` muestra algo en pantalla; `return` entrega un valor al código que llamó a la función, que puede guardarlo, compararlo o usarlo en otra cuenta. Las funciones que devuelven valores son mucho más reutilizables y fáciles de probar.

El texto entre triples comillas al inicio del cuerpo es la docstring: documenta qué hace la función y aparece en la ayuda (`help(funcion)`).

Ejemplo

```
1 def saludar(nombre, saludo="Hola"):
2     """Devuelve un saludo personalizado."""
3     return f"{saludo}, {nombre}!"
4
5 print(saludar("Leo"))
6 print(saludar("Ana", saludo="Buenas"))
7 print(saludar.__doc__)
```

Salida real

```
Hola, Leo!
Buenas, Ana!
Devuelve un saludo personalizado.
```

El ejemplo, línea por línea

1. `saludo="Hola"` es un parámetro con valor por defecto.
2. `return` devuelve el texto armado con la f-string.
3. La segunda llamada reemplaza el saludo usando el nombre del parámetro.
4. `saludar.__doc__` muestra la docstring.

Consejos

- Una función, una responsabilidad. Si su nombre necesita 'y', probablemente son dos funciones.
- Nunca uses listas o dicts como valor por defecto (`def f(x=[])`): usa `None`.

Ejercicio

Escribe una función `es_primo(n)` y úsala para imprimir los primos del 1 al 50.

Errores frecuentes

Error frecuente	Cómo resolverlo
La función devuelve <code>None</code>	Te faltó <code>return</code> , o usaste <code>print</code> dentro de la función en lugar de devolver el valor.
<code>TypeError: missing 1 required positional argument</code>	Llamaste a la función con menos argumentos de los que pide.

Práctica evaluable

Las funciones del juego · Pieza del proyecto: Capítulo 10: las funciones del juego

Ahora sí: convierte la lógica del juego en funciones reutilizables. El motor del juego las usará.

- `evaluar(secreto, intento)`: devuelve "Más alto", "Más bajo" o "¡Correcto!".
- `es_valido(numero, minimo=1, maximo=100)`: devuelve `True` si el número está dentro del rango, extremos incluidos.
- `mensaje_final(intentos)`: devuelve "¡Ganaste en 1 intento!" o, si son varios, "¡Ganaste en 3 intentos!".

Las tres deben usar `return`, no `print`.

Plantilla para empezar

```

1 def evaluar(secreto, intento):
2     """Devuelve "Más alto", "Más bajo" o "¡Correcto!"."""
3     if intento < secreto:
4         return "Más alto"
5     # completa los otros dos casos
6
7
8 def es_valido(numero, minimo=1, maximo=100):
9     """True si numero está entre minimo y maximo (incluidos)."""
10
11
12 def mensaje_final(intentos):
13     """Singular para 1 intento, plural para varios."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué devuelve una función sin return?

- a) Error
- b) 0
- c) None
- d) ""

11 Errores y cómo leerlos

Los errores son información, no fracasos. Los más comunes: `SyntaxError` (código mal escrito), `NameError` (variable inexistente), `TypeError` (tipos incompatibles), `ValueError` (valor inválido) e `IndexError` (índice fuera de rango).

Lee el traceback de abajo hacia arriba: la última línea dice el tipo de error y el mensaje; las anteriores dicen dónde ocurrió.

`try/except` permite atrapar un error y reaccionar en lugar de que el programa se detenga.

El concepto a fondo

Un mensaje de error (traceback) se lee de abajo hacia arriba: la última línea dice el tipo de error y una descripción; las líneas anteriores muestran el camino, con el archivo y el número de línea donde ocurrió.

Los errores más comunes: `SyntaxError` (código mal escrito), `NameError` (nombre inexistente), `TypeError` (operación con tipos que no combinan), `ValueError` (valor del tipo correcto pero inválido, como `int("hola")`), `IndexError` y `KeyError` (posición o clave inexistente) y `ZeroDivisionError`.

`try / except` atrapa un error para que el programa no se caiga: Python intenta el bloque `try` y, si ocurre el error indicado, ejecuta el bloque `except`. Atrapa siempre un tipo concreto (`except ValueError:`); un `except :` vacío esconde errores que deberías ver.

Validar datos que escribe una persona es el uso más típico: intentas convertir y, si falla, avisas y devuelves algo que indique "inválido", como `None`.

Ejemplo

```

1  try:
2      numero = int("diez")
3  except ValueError as e:
4      print("Ups:", e)
5
6  lista = [1, 2, 3]
7  try:
8      print(lista[10])
9  except IndexError:
10     print("Ese índice no existe")

```

Salida real

```

Ups: invalid literal for int() with base 10: 'diez'
Ese índice no existe

```

El ejemplo, línea por línea

1. `int("diez")` lanza `ValueError`; el `except` lo atrapa y muestra el mensaje guardado en `e`.
2. `lista[10]` lanza `IndexError`, que se atrapa en el segundo `try`.

Consejos

- Atrapa errores específicos (`except ValueError`), nunca un `except:` vacío.
- Anota en tu cuadernillo cada error nuevo que encuentres y cómo lo resolviste.

Ejercicio

Mejora la calculadora de propinas para que vuelva a pedir el monto si el usuario escribe algo inválido.

Errores frecuentes

Error frecuente	Cómo resolverlo
El <code>except</code> nunca se ejecuta	Estás atrapando otro tipo de error. Revisa en el traceback qué tipo ocurre realmente.
Un <code>except</code> vacío esconde todo	<code>except:</code> sin tipo atrapa incluso tus propios errores de código. Escribe siempre el tipo.

Práctica evaluable

A prueba de errores · Pieza del proyecto: Capítulo 11: a prueba de errores

El último capítulo protege el juego contra entradas inválidas. Escribe `pedir_numero(texto, minimo=1, maximo=100)`, que recibe lo que escribió el jugador y:

- devuelve el número entero si es válido y está dentro del rango;
- devuelve `None` si no es un número entero ("hola", "3.5", "") o si está fuera de rango.

Usa `try / except ValueError` para la conversión.

Plantilla para empezar

```
1 def pedir_numero(texto, minimo=1, maximo=100):
2     """Devuelve el número si es válido, o None si no lo es."""
3     try:
4         numero = int(texto)
5     except ValueError:
6         return None
7     # ahora revisa el rango
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

print(edad) sin haber definido edad lanza...

- a) SyntaxError
- b) NameError
- c) TypeError
- d) ValueError

Proyecto integrador

Adivina el número

Un juego de consola donde la computadora elige un número secreto y el jugador intenta adivinarlo con pistas.

Paso a paso

1. Importa random y genera un número entre 1 y 100 con random.randint.
2. Usa un bucle while para pedir intentos con input() y conviértelos a int dentro de try/except.
3. Da pistas: 'más alto' o 'más bajo'. Cuenta los intentos.
4. Al acertar, muestra los intentos usados y pregunta si quiere jugar otra vez.
5. Extra: niveles de dificultad (rango y número máximo de intentos).

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Puedo explicar y usar: tu primer programa
- ☐ Puedo explicar y usar: variables y tipos de datos
- ☐ Puedo explicar y usar: operadores
- ☐ Puedo explicar y usar: cadenas de texto (strings)
- ☐ Puedo explicar y usar: entrada del usuario y conversiones
- ☐ Puedo explicar y usar: condicionales
- ☐ Puedo explicar y usar: bucles for y while
- ☐ Puedo explicar y usar: listas y tuplas
- ☐ Puedo explicar y usar: diccionarios y conjuntos
- ☐ Puedo explicar y usar: funciones
- ☐ Puedo explicar y usar: errores y cómo leerlos
- ☐ Aprobé las 11 prácticas evaluables y mi proyecto ejecuta la demo
- ☐ Terminé el proyecto "Adivina el número"

Tu proyecto del nivel, pieza a pieza

Adivina el número

Tu primer programa completo. Cada lección del Nivel 1 agrega un capítulo: primero la bienvenida, luego los cálculos, las pistas, una computadora que juega sola y, al final, las funciones y la protección contra errores que hacen funcionar el juego de verdad.

Cada práctica evaluable del nivel es una pieza de este programa. En la app, la sección **Mi proyecto** (desde el Árbol o el Inicio) une tus soluciones aprobadas en un solo archivo, `adivina_el_numero.py`, te deja ejecutar la demo en el navegador cuando están todas y descargarlo en .zip (con un README) para abrirlo en tu IDE. Las piezas pendientes se reemplazan por versiones provisionales, así que puedes descargarlo en cualquier momento.

Las piezas

Lección	Pieza	Práctica
1.1 Tu primer programa	Capítulo 1: la bienvenida	La bienvenida del juego
1.2 Variables y tipos de datos	Capítulo 2: la configuración	La configuración del juego
1.3 Operadores	Capítulo 3: los cálculos	Los cálculos del juego
1.4 Cadenas de texto (strings)	Capítulo 4: los mensajes	Los mensajes del juego
1.5 Entrada del usuario y conversiones	Capítulo 5: la entrada del jugador	Convertir lo que escribe el jugador
1.6 Condicionales	Capítulo 6: las pistas	Las pistas del juego
1.7 Bucles for y while	Capítulo 7: la computadora juega	La computadora adivina
1.8 Listas y tuplas	Capítulo 8: el historial	El historial de intentos
1.9 Diccionarios y conjuntos	Capítulo 9: los récords	Récords y repetidos
1.10 Funciones	Capítulo 10: las funciones del juego	Las funciones del juego
1.11 Errores y cómo leerlos	Capítulo 11: a prueba de errores	A prueba de errores

El programa principal

Este es el código que Pythonia agrega al final del archivo: no tienes que escribirlo, pero vale la pena leerlo, porque muestra cómo se conectan tus piezas. `demo()` hace un recorrido automático y el bloque `if __name__ == "__main__":` es lo que corre al ejecutar el archivo en tu IDE.

```

1  # ===== Motor del juego (escrito por Pythonia, usa tus funciones)
   ↩ =====
2  import random
3
4
5  def demo():
6      """Partida automática: muestra tu juego funcionando sin usar el teclado."""
7      print("— Tus capítulos —")
8      for capitulo in CAPITULOS:
9          print("\n►", capitulo.__doc__)
10         capitulo()
11     print("\n— Partida de demostración (el secreto es 37) —")
12     intentos = 0
13     for texto in ["50", "hola", "200", "25", "37"]:
14         numero = pedir_numero(texto)
15         if numero is None:
16             print(f"'{texto}' no vale: escribe un número del 1 al 100")
17             continue
18         intentos += 1
19         pista = evaluar(37, numero)
20         print(f"{numero} -> {pista}")
21         if pista == "¡Correcto!":
22             print(mensaje_final(intentos))
23             break
24
25
26  def jugar():
27      """Partida real con teclado: ejecútala en tu IDE."""
28      secreto = random.randint(1, 100)
29      print("=== ADIVINA EL NÚMERO ===")
30      intentos = 0
31      while intentos < 7:
32          numero = pedir_numero(input("Tu número: "))
33          if numero is None:
34              print("Escribe un número entero del 1 al 100.")
35              continue
36          intentos += 1
37          pista = evaluar(secreto, numero)
38          print(pista)
39          if pista == "¡Correcto!":
40              print(mensaje_final(intentos))
41              return
42      print(f"Se acabaron los intentos. Era el {secreto}.")
43
44
45  if __name__ == "__main__":
46      opcion = input("¿Ver la demo (d) o jugar (j)? ").strip().lower()
47      if opcion == "d":
48          demo()
49      else:
50          jugar()

```

Solucionario de autoevaluación

- 1. Tu primer programa:** a) AB. sep define qué va entre los valores. Con sep="" no se pone nada.
- 2. Variables y tipos de datos:** d) float. La división con / siempre devuelve float en Python 3, aunque el resultado sea exacto.
- 3. Operadores:** b) 1. % devuelve el resto: $10 = 3 \times 3 + 1$.
- 4. Cadenas de texto (strings):** c) "ol". Incluye el índice 1 (o) y excluye el 3, así que toma o y l.
- 5. Entrada del usuario y conversiones:** c) str. input() siempre devuelve str. Convertir es tu responsabilidad.
- 6. Condicionales:** d) B. 0 es falsy, por eso entra al else.
- 7. Bucles for y while:** d) 4. Genera 2, 4, 6, 8. El 10 no se incluye.
- 8. Listas y tuplas:** c) TypeError. Las tuplas son inmutables: no admiten asignación por índice.
- 9. Diccionarios y conjuntos:** b) 3. Un set elimina duplicados: queda {1, 2, 3}.
- 10. Funciones:** c) None. Toda función devuelve algo; si no hay return, devuelve None.
- 11. Errores y cómo leerlos:** b) NameError. NameError: el nombre no existe en el ámbito actual.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. Tu primer programa · La bienvenida del juego

Pista 1: Cada `print` produce una línea nueva.

Pista 2: Para la línea 4: `print("Más alto", "Más bajo", "¡Correcto!", sep=" | ")`.

```
1 print("=== ADIVINA EL NÚMERO ===")
2 print("Pensé un número del 1 al 100.")
3 print("¿Podrás descubrirlo?")
4 print("Más alto", "Más bajo", "¡Correcto!", sep=" | ")
```

Por qué funciona: Tres `print` simples y uno con tres argumentos. Por defecto `print` separa los argumentos con un espacio; con `sep=" | "` usa la barra rodeada de espacios.

2. Variables y tipos de datos · La configuración del juego

Pista 1: Un número entero se escribe sin comillas: `minimo = 1`.

Pista 2: `print(f"{nombre_juego}: del {minimo} al ...")`

```
1 nombre_juego = "Adivina el número"
2 minimo = 1
3 maximo = 100
4 intentos_max = 7
5 con_pistas = True
6 print(f"{nombre_juego}: del {minimo} al {maximo} en {intentos_max} intentos")
```

Por qué funciona: Cada variable guarda un valor con su tipo natural. La f-string arma la frase leyendo las variables, así que si mañana cambias `maximo = 500`, el mensaje se actualiza solo.

3. Operadores · Los cálculos del juego

Pista 1: Los paréntesis importan: `(minimo + maximo) // 2`.

Pista 2: Una comparación ya es un booleano: `acerto = intento == secreto` guarda `True` o `False`.

```
1 cantidad = maximo - minimo + 1
2 mitad = (minimo + maximo) // 2
3 distancia = abs(secreto - intento)
4 es_par = secreto % 2 == 0
5 acerto = intento == secreto
```

Por qué funciona: `//` da la mitad entera; `abs` elimina el signo, así la distancia es la misma si el intento se pasa o se queda corto. `secreto % 2 == 0` y `intento == secreto` son comparaciones: su resultado es directamente `True` o `False`.

4. Cadenas de texto (strings) · Los mensajes del juego

Pista 1: Los métodos se encadenan: `texto.strip().title()`.

Pista 2: La primera posición es la 0: `jugador[0]`.

```
1 jugador = jugador_crudo.strip().title()
2 mensaje = f"{jugador}, intento {numero_intento} de {intentos_max}"
3 inicial = jugador[0]
4 grito = mensaje.upper()
5 largo = len(jugador)
```

Por qué funciona: Primero se limpia el dato (`strip` y `title`) y recién después se usa: así todo lo demás, incluido `largo`, trabaja sobre el nombre correcto. Es un hábito clave al procesar lo que escribe una persona.

5. Entrada del usuario y conversiones · Convertir lo que escribe el jugador

Pista 1: `int()` ignora los espacios de los bordes.

Pista 2: Para unir texto y número con `+`, convierte el número: `str(intento)`.

```
1 intento = int(texto)
2 doble = intento * 2
3 siguiente = intento + 1
4 resumen = "Elegiste el " + str(intento)
```

Por qué funciona: Convertir una sola vez, al principio, deja todo el resto del código trabajando con números de verdad. `str()` hace el camino inverso cuando necesitas pegar el número a un texto con `+` (con una f-string no haría falta).

6. Condicionales · Las pistas del juego

Pista 1: Con dos comparaciones (`<` y `>`) el `else` queda para cuando son iguales.

Pista 2: `cerca = distancia > 0 and distancia <= 5` también funciona.

```
1 if intento < secreto:
2     pista = "Más alto"
3 elif intento > secreto:
4     pista = "Más bajo"
5 else:
6     pista = "¡Correcto!"
7
8 distancia = abs(secreto - intento)
9 cerca = 0 < distancia <= 5
```

Por qué funciona: Tres caminos excluyentes: menor, mayor o igual. Para `cerca` hay dos condiciones a la vez (no acertó y está a 5 o menos), que la comparación encadenada `0 < distancia <= 5` expresa en una sola línea.

7. Bucles for y while · La computadora adivina

Pista 1: Necesitas un `if` / `elif` / `else` dentro del `while`, con la misma sangría que el `print`.

Pista 2: Si te sale "más de 100.000 vueltas", es que adivinado nunca cambia: falta el `else: adivinado = medio`.

```

1 bajo = minimo
2 alto = maximo
3 pasos = 0
4 adivinado = None
5 while adivinado != secreto:
6     medio = (bajo + alto) // 2
7     pasos = pasos + 1
8     print("Pruebo", medio)
9     if medio < secreto:
10        bajo = medio + 1
11    elif medio > secreto:
12        alto = medio - 1
13    else:
14        adivinado = medio

```

Por qué funciona: Cada vuelta descarta la mitad de los números que quedan: 100, 50, 25, 12, 6, 3, 1. Por eso 7 intentos alcanzan siempre. Esta idea se llama búsqueda binaria y es uno de los algoritmos más importantes de la informática.

8. Listas y tuplas · El historial de intentos

Pista 1: Los índices negativos cuentan desde el final: `-1` es el último.

Pista 2: `sorted()` devuelve una lista nueva; `.sort()` cambia la original.

```

1 cantidad = len(historial)
2 primero = historial[0]
3 ultimo = historial[-1]
4 ultimos_tres = historial[-3:]
5 ordenados = sorted(historial)
6 extremos = (min(historial), max(historial))

```

Por qué funciona: Todo se resuelve con índices, rebanados y funciones integradas, sin bucles. `sorted` protege los datos originales, algo que vas a agradecer cuando un mismo dato se use en varios lugares del programa.

9. Diccionarios y conjuntos · Récordeos y repetidos

Pista 1: Arranca con `mejor = None` y reemplázalo cuando encuentres alguien con menos intentos.

Pista 2: Un set no tiene repetidos: si quedó más chico que la lista, había repetidos.

```

1 if jugador not in records or intentos < records[jugador]:
2     records[jugador] = intentos
3
4 mejor = None
5 for nombre, n in records.items():
6     if mejor is None or n < records[mejor]:
7         mejor = nombre
8
9 repetidos = len(set(historial)) < len(historial)
10 unicos = sorted(set(historial))

```

Por qué funciona: El `or` se evalúa de izquierda a derecha: si el jugador es nuevo, ni siquiera se lee `records[jugador]`, y así se evita el `KeyError`. El patrón de "el mejor hasta ahora" (`mejor = None` y comparar en cada vuelta) aparece en muchísimos programas.

10. Funciones · Las funciones del juego

Pista 1: Como `return` termina la función, después de dos `if` con `return` el tercer caso puede ir sin condición.

Pista 2: `return minimo <= numero <= maximo` devuelve directamente el booleano.

```

1 def evaluar(secreto, intento):
2     """Devuelve "Más alto", "Más bajo" o "¡Correcto!"."""
3     if intento < secreto:
4         return "Más alto"
5     if intento > secreto:
6         return "Más bajo"
7     return "¡Correcto!"
8
9
10 def es_valido(numero, minimo=1, maximo=100):
11     """True si numero está entre minimo y maximo (incluidos)."""
12     return minimo <= numero <= maximo
13
14
15 def mensaje_final(intentos):
16     """Singular para 1 intento, plural para varios."""
17     if intentos == 1:
18         return "¡Ganaste en 1 intento!"
19     return f"¡Ganaste en {intentos} intentos!"

```

Por qué funciona: Cada función hace una sola cosa y devuelve su resultado. Así el motor del juego puede combinarlas, y el evaluador (como harás tú con `pytest` en el Nivel 2) puede probarlas por separado con distintos valores.

11. Errores y cómo leerlos · A prueba de errores

Pista 1: La conversión puede fallar; la comparación del rango no. Por eso solo `int(texto)` va dentro del `try`.

Pista 2: Si la función termina sin `return`, devuelve `None` sola, pero escribirlo explícito es más claro.

```

1 def pedir_numero(texto, minimo=1, maximo=100):
2     """Devuelve el número si es válido, o None si no lo es."""
3     try:
4         numero = int(texto)
5     except ValueError:
6         return None
7     if minimo <= numero <= maximo:
8         return numero
9     return None

```

Por qué funciona: Dentro del `try` va solo lo que puede fallar. Si la conversión funciona, se revisa el rango. Devolver `None` permite que el motor del juego pregunte `if numero is None` y vuelva a pedir el dato sin caerse.